Mean Reversion Strategy: Examining the performance efficiency of reversal pattern in Tesla Stock.

by jagteshwar singh

Quantinsti, Epat Project

January 10, 2025

Project Abstract

This paper examines the performance of a mean reverting strategy applied on intraday 1 minute data to Tsla stock. Unlike Other traditional papers which focus on mean reverting returns of stock prices we focus on mean reverting prices and assess their performance on 10 years of intraday Tesla data. In this paper we generate contrarian trading strategy using percentage drop in the asset price of tsla stock within the first hour of trading. This strategy does not discriminate against the current state of the long-term trend in the stock.

Project motivation / Introduction

Mean reversion strategy is used in various domains with primarily in the statistical arbitrage of two cointegrated pairs. The diverse application of this theory makes it suitable for assessing the discrepancy between the ideal price of the stock and the extreme values. In our research we have used the fundamental basis of mean reversion to examine the results for creating a long-based Trading strategy on 1-minute Tesla stock data. This paper evaluates the results of the strategy operated between the regular trading hours. In this paper we first examine the comparison of the daily log return of the TSLA stock to gaussian distribution of the dataset having minimum, maximum and mean equivalent to the TSLA daily dataset. We additionally run QQ-plot test and Jarque bera test to decipher the distribution of the TSLA daily log returns. The focus of the test is to see the deviation of TSLA returns to the normally distributed return. To understand the behaviour of trends on the price data we have also incorporated the use of custom function which differentiates the trend based on the 20 day moving average data. This function provides additional information to behaviour of bearishness and bullishness in stock in uptrend and downtrend. Since a leptokurtic distribution would result in fatter tails and high peaks it would mean more extreme nature of stock returns. In this paper we have focused to develop mean reverting strategy on extreme drops in the tsla stock within the first hour of trading. To measure the extreme downward

moves, we chose 4% drop in the very first hour of trading as the cutoff value. The cutoff value uses the most recent high and most recent low with most recent low time> most recent high.

$$percentage\ drop = (Recent\ High - Recent\ Low)/(Recent\ High)$$

```python
# Function to find the distribution of returns
def returns_distribution(data, year_wise = None):
    """ The function uses data as input which can be rendered via glob library
        the function calculates and plots distribution of returns plot overlaid by
        pdf for normally distributed data. For pdf we use min, max, mean and standard deviation
        to plot.

        Additionally to check normality we also use jarque bera test to validate if the series
        is normally distributed or not"""
    try:
        df = pd.read_csv(data, parse_dates = True, skipfooter = 1, index_col = 0, engine = "python") # here one can iterate through files using glob
        df["log_returns"] = np.log(df.Last / df.Last.shift(1))
        df.dropna(inplace = True)
        log_returns_range = np.linspace(min(df["log_returns"]), max(df["log_returns"]), num = 1000) # max and min values of log_returns
        mean = df["log_returns"].mean() # mean of log returns
        std_dev = df["log_returns"].std() # standard deviation of log returns
        norm_pdf = scs.norm.pdf(log_returns_range, loc = mean, scale = std_dev) # using scipy we make pdf function using max, min range, std, mean
        fig, ax = plt.subplots(1,2, figsize = (15,7.5))
        sns.distplot(df["log_returns"], kde = False, norm_hist = True, ax = ax[0]) # norm_hist -> to make area of histogram -> 1
        ax[0].set_title("Distribution of Tsla returns", fontsize = 15)
        ax[0].plot(log_returns_range, norm_pdf, color = "Red", linewidth = 2, label =f"N({mean:.4f},{std_dev**2:.5f})")
        ax[0].legend(loc="best")
        # Q-Q plot analysis
        sm.qqplot(df["log_returns"], line = "q", ax = ax[1])
        ax[1].set_title("Q-Q plot", fontsize = 15)
        plt.show()
    except Exception as e:
        print("Could not compute the normality test for the given ticker", e)

    def jarque_bera_test():
        try:
            jb_stat, p_value, skewness, Kurtosis = jarque_bera(df["log_returns"])
            print(f"The T stat value for the Tsla returns is {jb_stat:.4f}, the P-Value is {p_value:.3f}, skewness:{skewness} and kurtosis of {Kurtosis}")
            print("="*100)
        except Exception as er:
            print("Could not compute Jarque bera test for the ticker", er)
    jarque_bera_test()
```

Figure 1 The returns distribution function tests whether the daily log returns exhibit gaussian curve or not, additionally qqplot and jarque bera are run to further solidify the results.

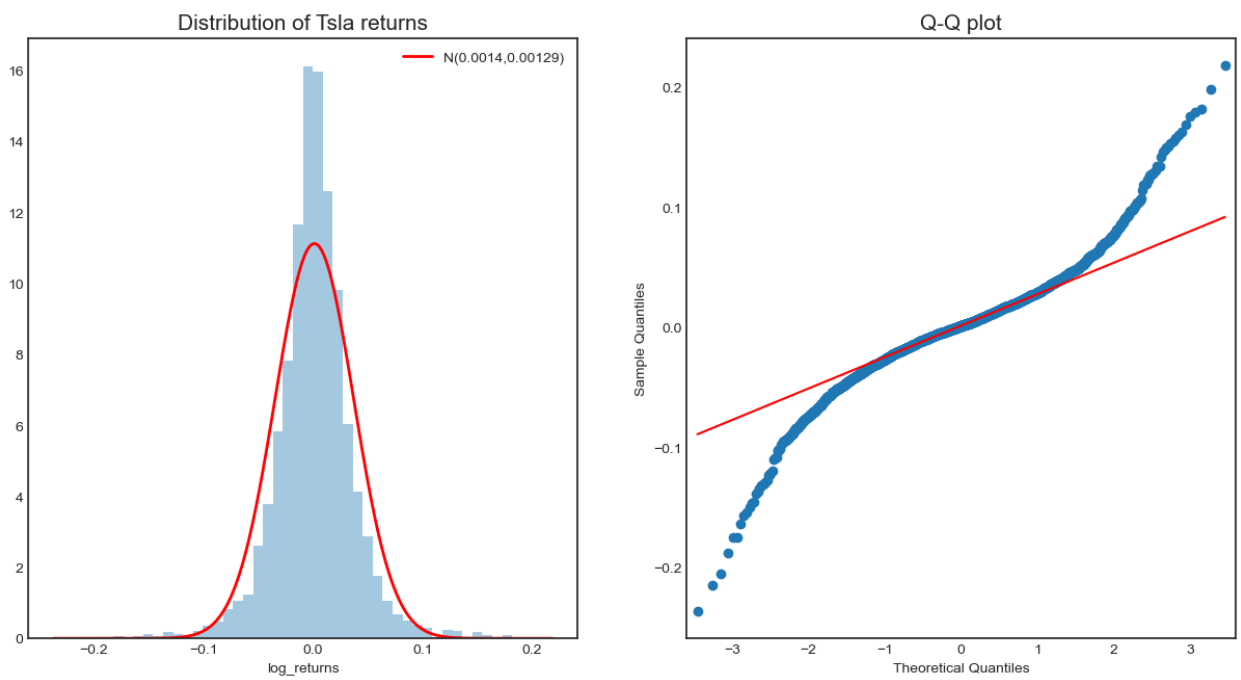


Figure 2: we see extreme values in the return distribution. Jarque Bera test results: The T stat value for the Tesla returns is 3573.0894, the P-Value is 0.000, skewness: -0.0035294508323746835 and kurtosis of 7.87454029687386

Statistics as per the classification of trend based on 20 day moving average. Daily candles above the 20 day sma will be considered under upwards trend and candles below the 20 day sma will be classified as under downtrend. If candle closes above the open price it is considered bullish candle and if candle closes below the open it is considered as bearish candle.

Up momentum stats: Mean return = 0.01, Highest return = 0.18, Lowest return = -0.14

Candle color

Bullish 1208

Bearish 841

Downward momentum stats: Mean return = -0.01, Highest return = 0.13, Lowest return = -0.22

Candle color

Bearish 976

Bullish 603

2025-01-09 16:43:09,038 - INFO - Upwards Momentum Bullish Log Returns Max: 0.18

2025-01-09 16:43:09,040 - INFO - Downwards Momentum Bullish Log Returns Max: 0.13

2025-01-09 16:43:09,042 - INFO - Upwards Momentum Bearish Log Returns Max: -0.14

2025-01-09 16:43:09,044 - INFO - Downwards Momentum Bearish Log Returns Max: -0.22

Data Mining

In this project we have used Barchart.com to retrieve 1 min Tesla stock data to develop our strategy. Since for any dataset bar chart allows 20,000 rows of data, we have explicitly used Glob library in python to converge 10 years of data sorted in ascending order. The concat function and sort values make sure the data is in ascending order for further processing.

```python
import glob
import pandas as pd
import matplotlib.pyplot as plt
import os

# Get all CSV files in the specified folder
csv_files = glob.glob(r"C:\Users\jay\Documents\Tsla_yearly_minute_data\*.csv")

# Function to optimize data
def data_optimise(combined_data):
    # Convert the "Time" column to datetime (pandas can handle ISO8601 format automatically)
    combined_data["Time"] = pd.to_datetime(combined_data["Time"], errors='coerce')

    # Sort data by "Time" in ascending order
    combined_data = combined_data.sort_values(by="Time", ascending=True).reset_index(drop=True)

    # # Drop unnecessary columns
    # combined_data.drop(columns=["Change", "%Chg"], inplace=True)

    # # Rename "Last" column to "Close"
    # combined_data.rename(columns={"Last": "Close"}, inplace=True)

    # Set "Time" as the index
    combined_data = combined_data.set_index("Time")

    # Return the cleaned DataFrame
    return combined_data

# Read and concatenate all CSV files, then apply the optimization function to each
combined_data = pd.concat([pd.read_csv(file).iloc[:-1] for file in csv_files], ignore_index=True)

# Apply data_optimise function to the concatenated DataFrame
combined_data = data_optimise(combined_data)

# Print the cleaned and optimized DataFrame
print(combined_data)

# # some checks to see if there are nan values in the datset
# print(combined_data.isna().sum()) #
output_directory = r"C:\Users\jay\Documents\Tsla_Combined_data"
file_name = "Tsla_Combined_1_min.csv"
output_file = os.path.join(output_directory, file_name)
combined_data.to_csv(output_file)
```

Figure 3: Custom function to aggregate the 1-minute data into a single file.

Data Analysis

In this project we ran preliminary data analysis to find the distribution of negative returns in the very first hour. We have analysed the drop of less than -4% for first trading hour for each year. This analysis computes the occurrence of the time the -4% drop was achieved, the total drop in the percentage price at 10:30. This analysis helps in deciphering the most repetitive number of percentage drop, thus helping to identify the outlier drops.

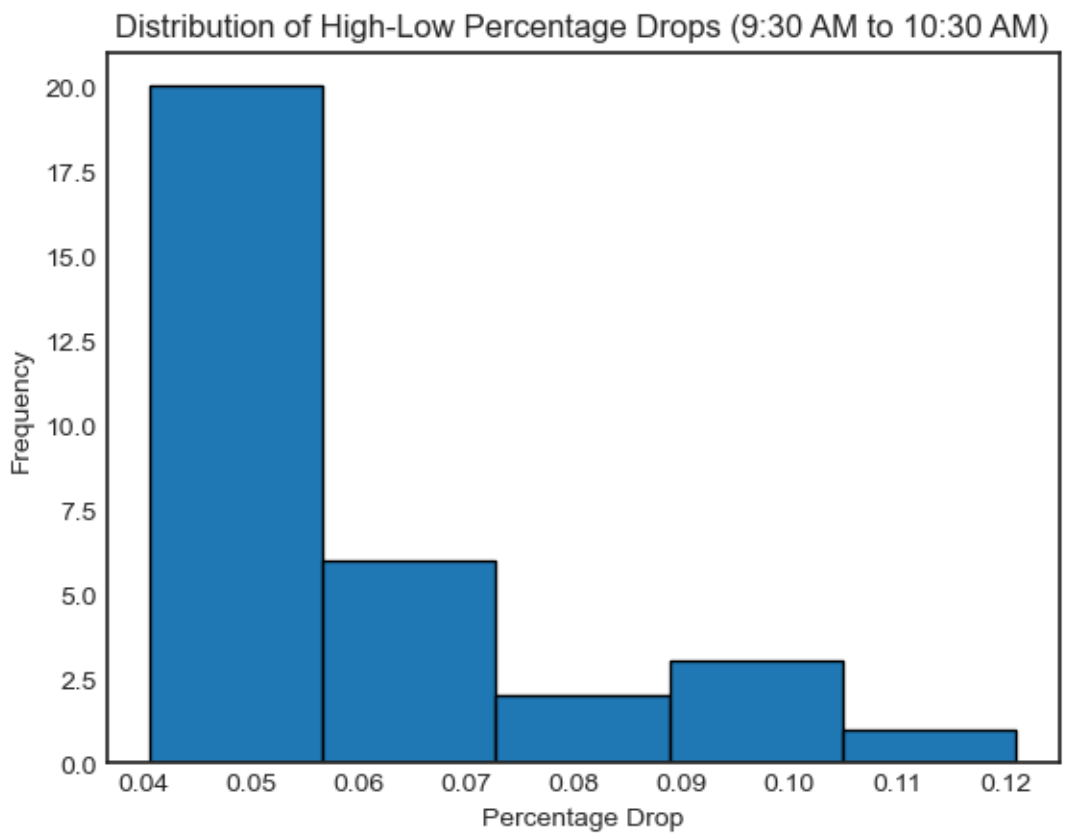


Figure 4 Frequency and drop percentage for year 2013

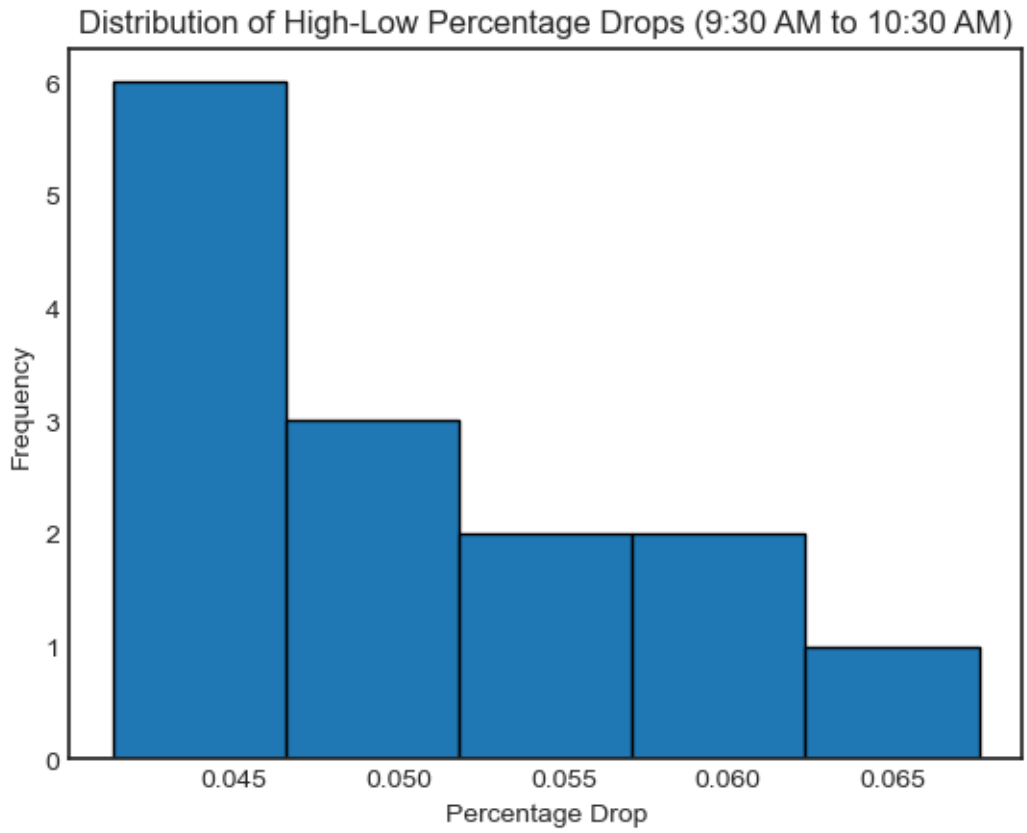


Figure 5 Frequency and drop percentage for year 2014

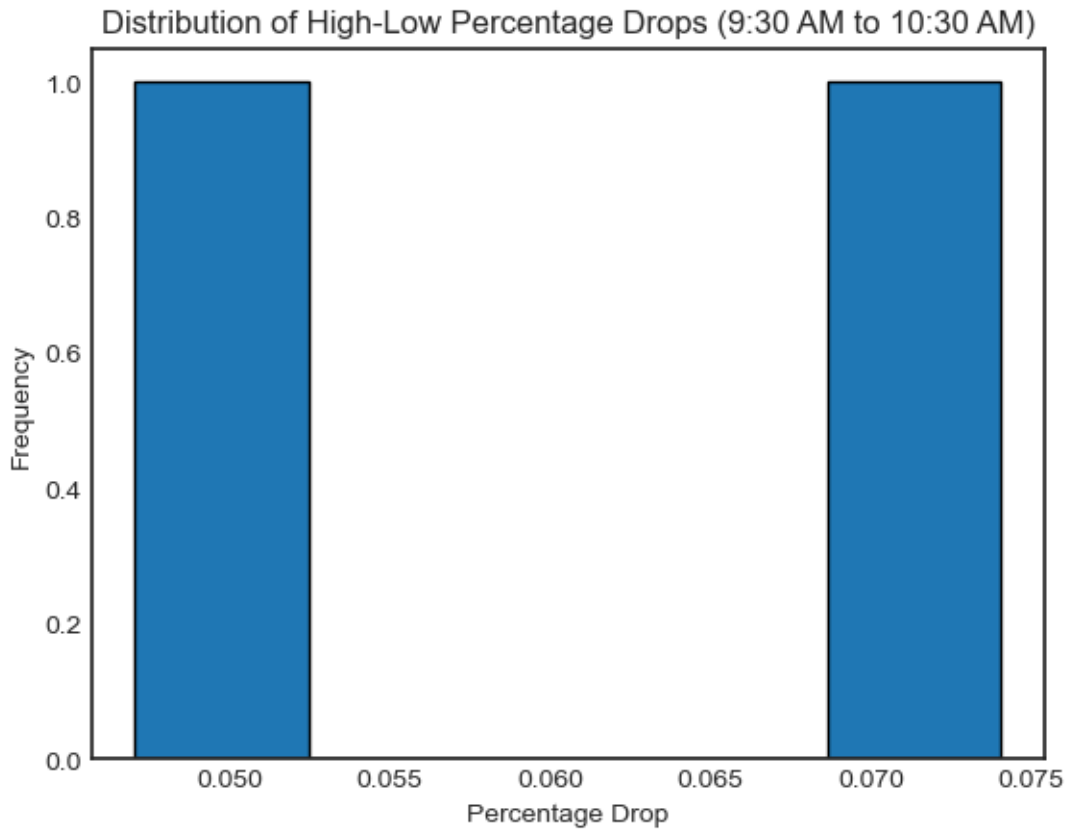


Figure 6 Frequency and drop percentage for year 2015

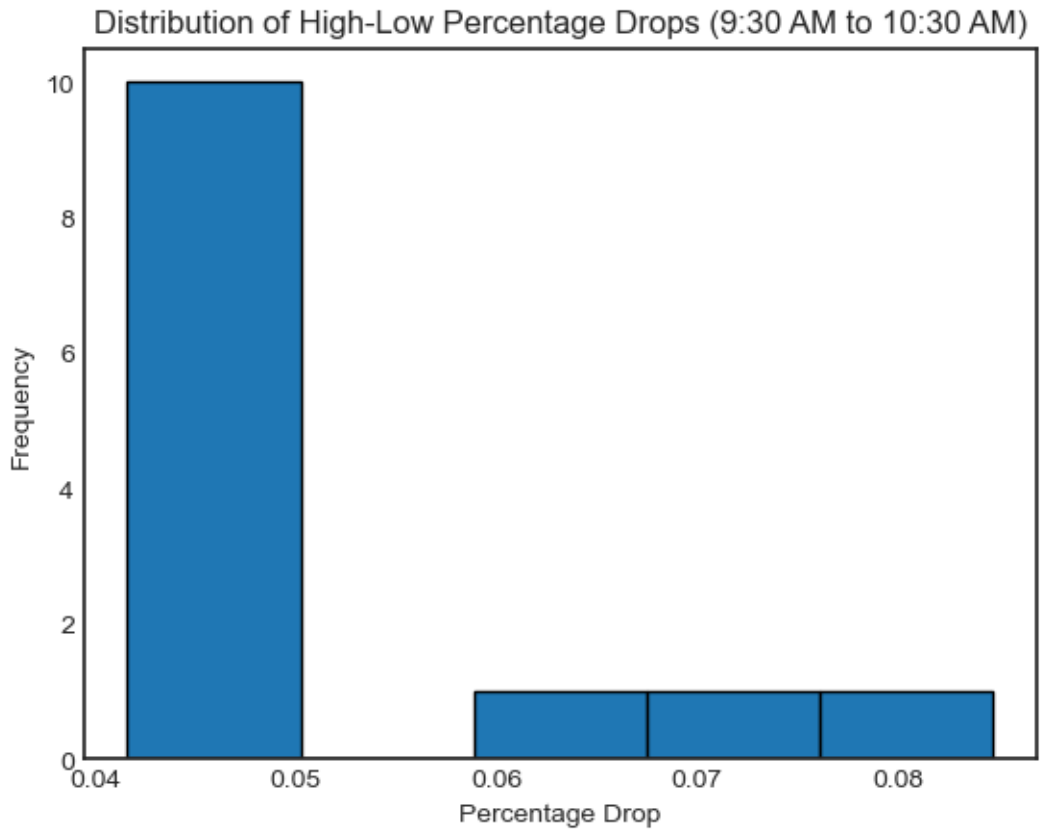


Figure 7 Frequency and drop percentage for year 2016

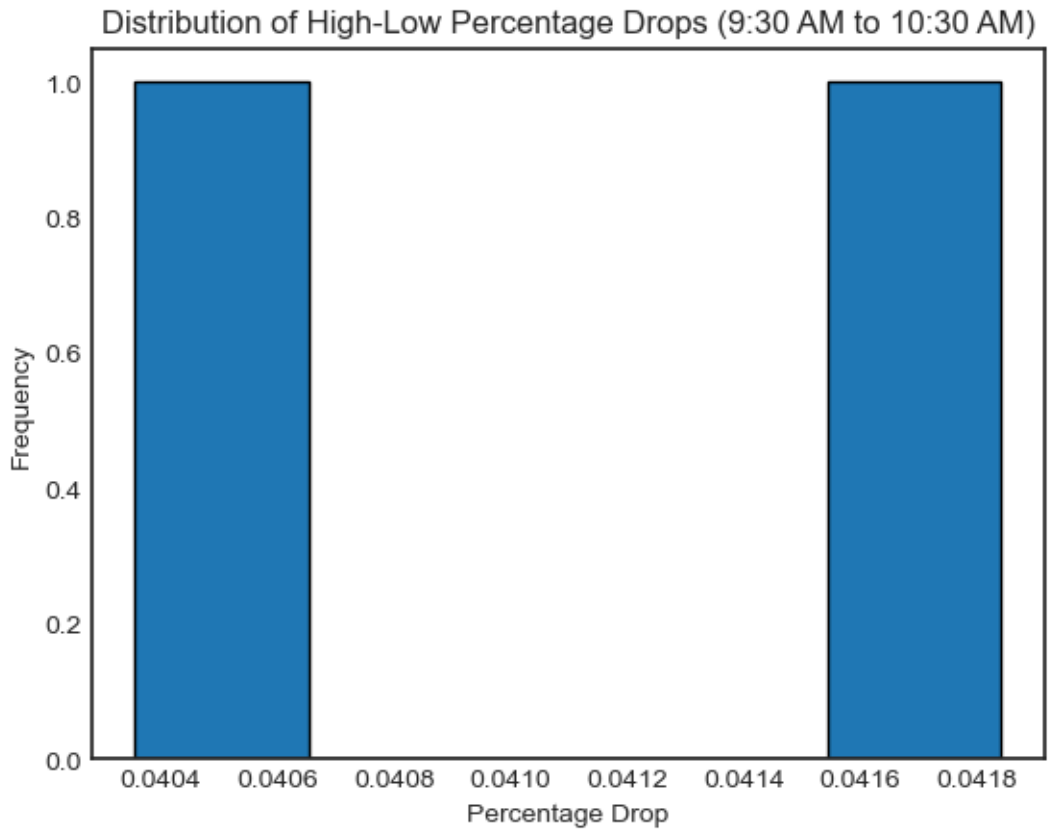


Figure 8 frequency and drop percentage for year 2017

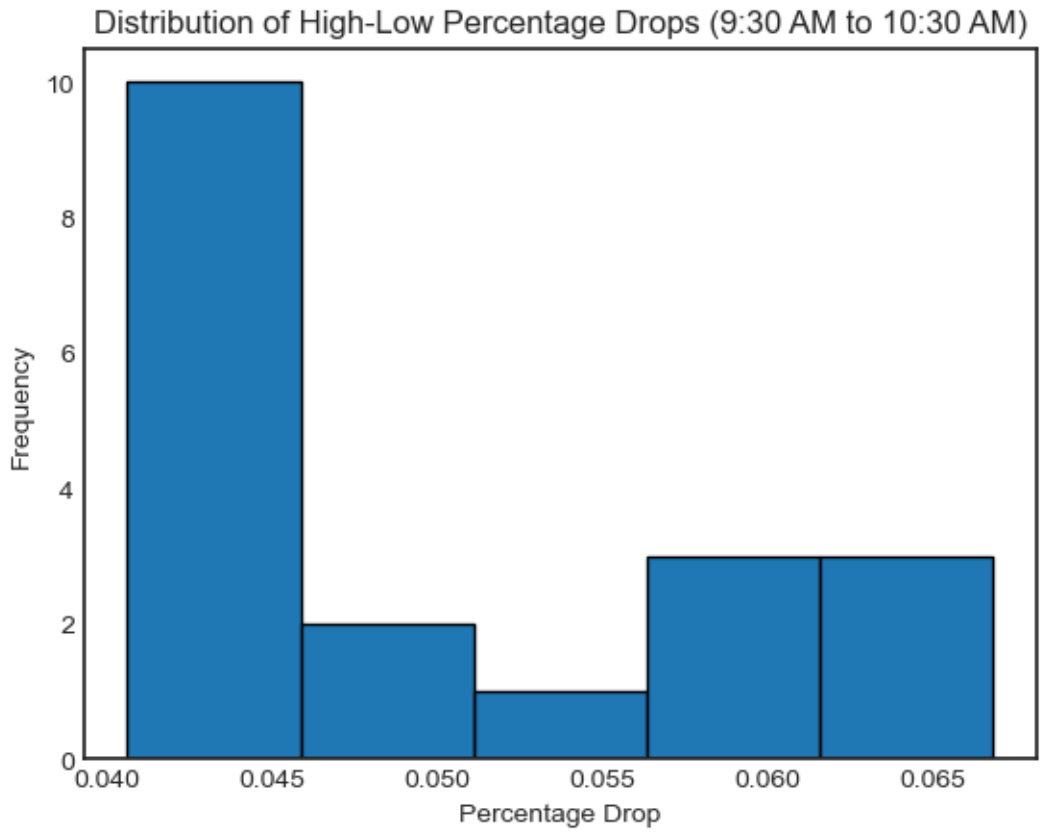


Figure 9 frequency and drop percentage for year 2018

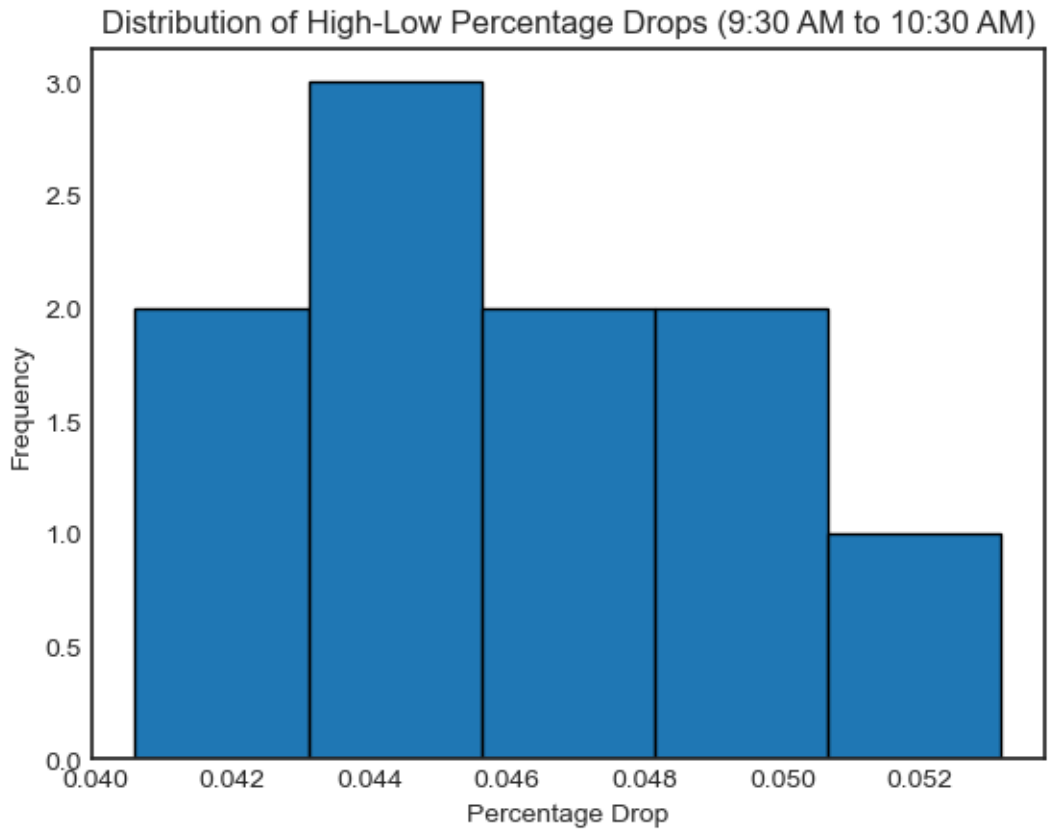


Figure 10 frequency and drop percentage for year 2019

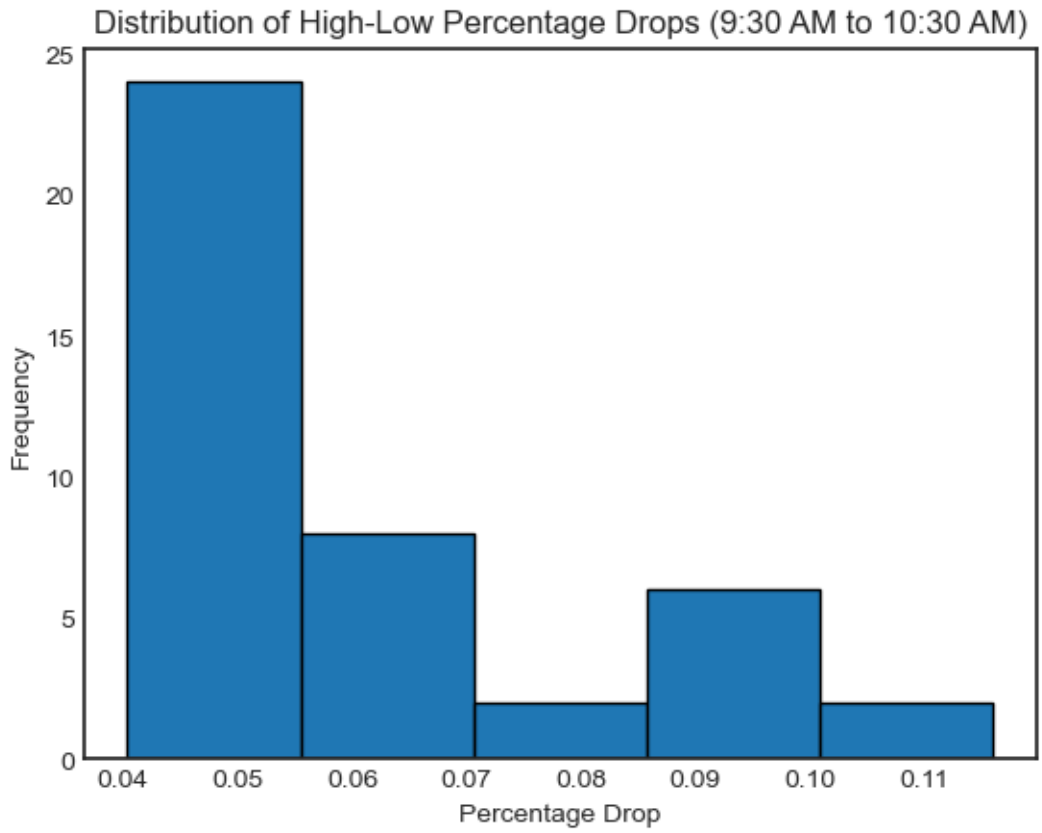


Figure 11 frequency and drop percentage for year 2020

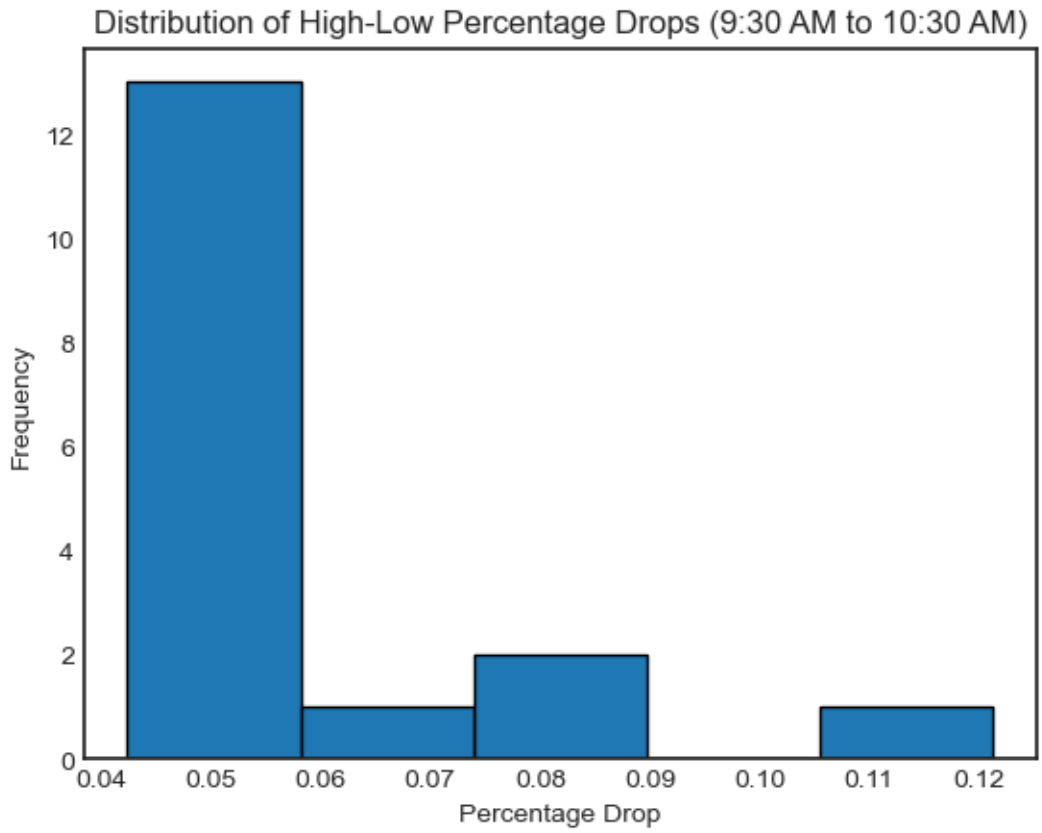


Figure 12 frequency and drop percentage for year 2021

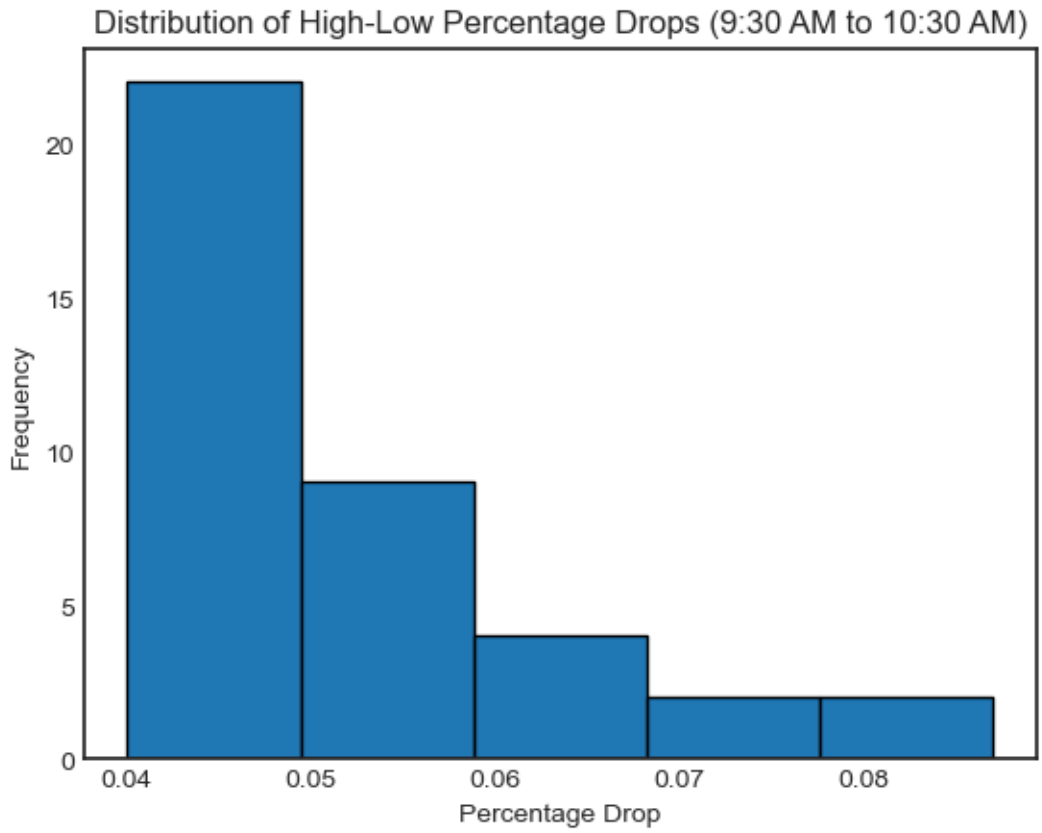


Figure 13 frequency and drop percentage for year 2022

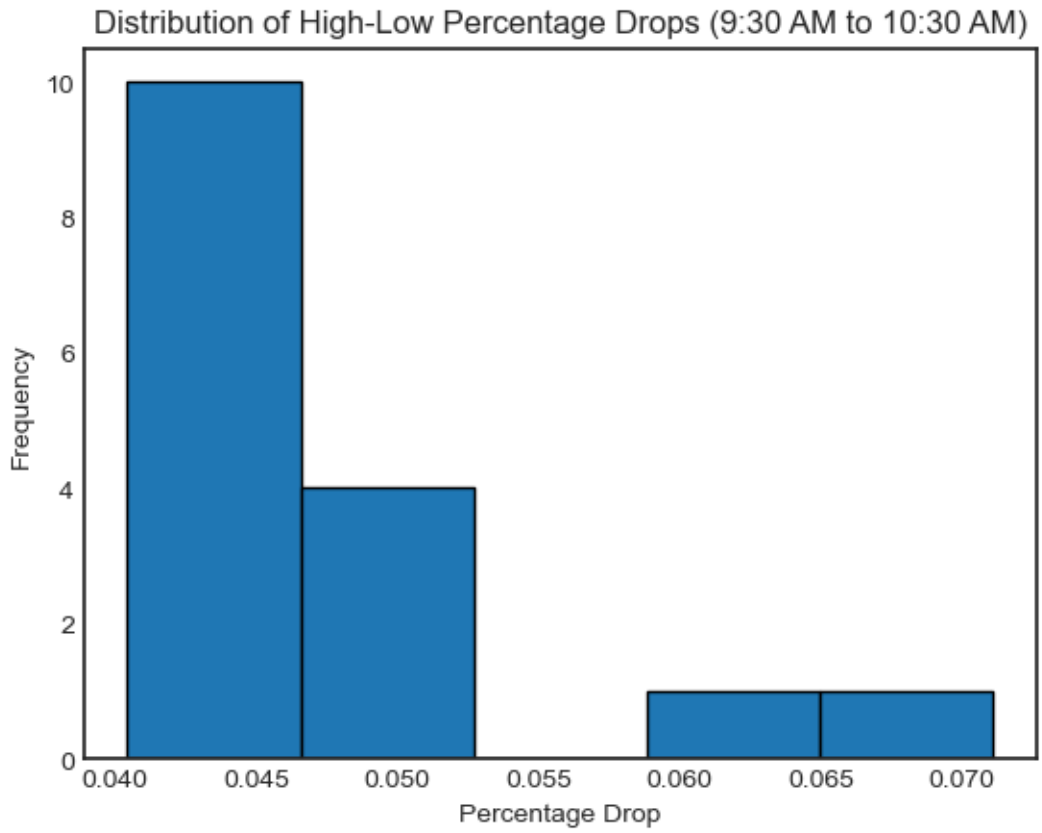


Figure 14 frequency and drop percentage for year 2023

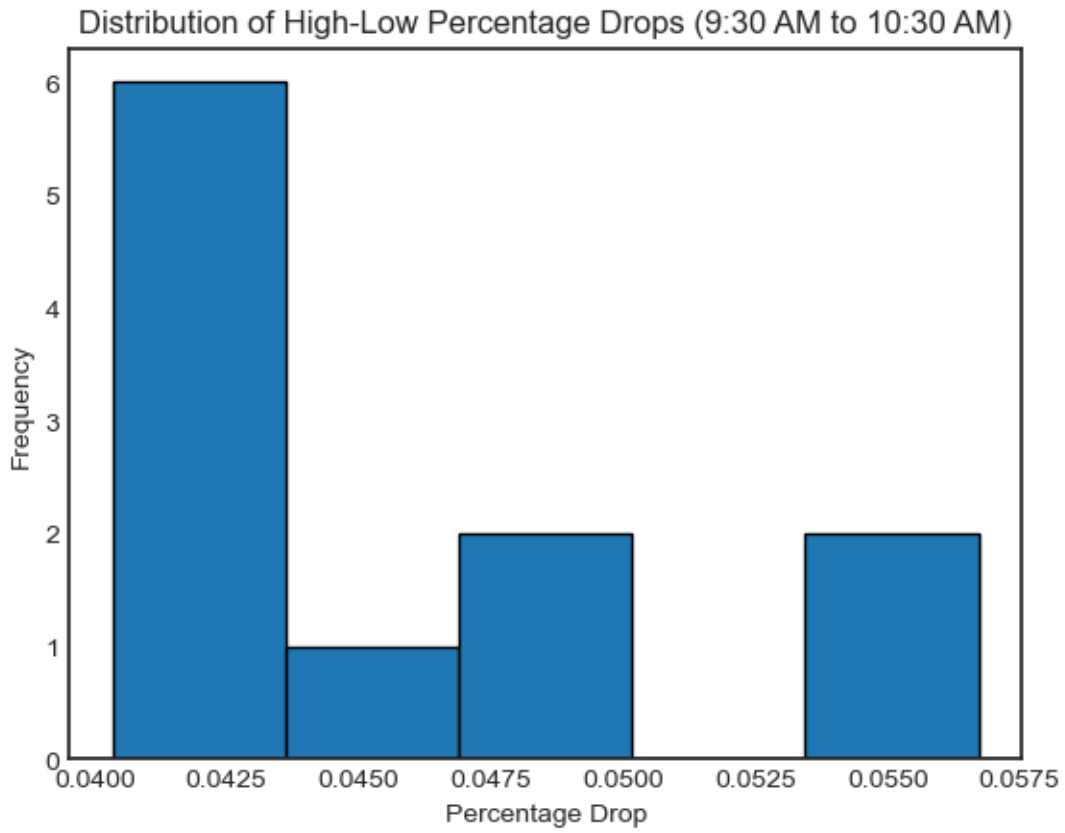


Figure 15 frequency and drop percentage for year 2024

The above graph shows the yearly drop in the intraday prices in terms of percentage. The graph shows the maximum drop achieved within 9.30 and 10:30 for Tesla stock on yearly basis. The above analysis shows that year 2015 and 2017 were very less volatile in terms of their drop percentage and frequency when compared to all the other years. Moreover, Tesla stock has maximum frequency in terms of occurrences in 4% to 5% drop. This analysis helped us in deciding the long criteria for entry. The long criteria is based on entry after 4% drop is achieved and setting stop loss at 6% and above which is achieved using the lower Bollinger band level in the event driven code.

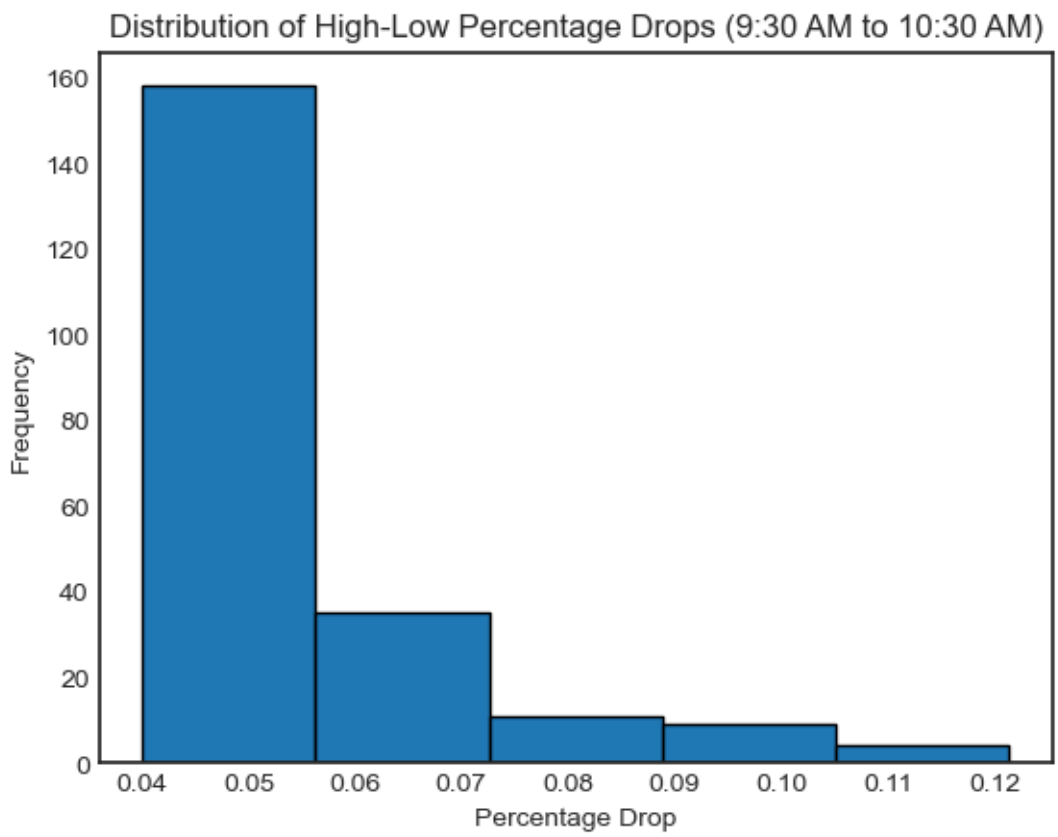


Figure 16 Overall Frequency distribution for 2013-2024.

Key findings

From the above graphical analysis, we see major barometer for percentage drop to be 4% to 6%. We also ran a test to find how many times the day end close was above the maximum low achieved during the first trading hour.

```
below_our_low = trade_data['Low Price<10:30']> trade_data["Close"]
print(below_our_low.value_counts())
# False    156
# True      61
# Name: count, dtype: int64
# so 156 times it closes above our low of 10:30
```

Figure 17 Results of the key findings

Out of total data set we see only 61 times the close was lower than the low achieved by 10:30 am. Since majority of the drops lie in the 4% to 6% , most of the trades show good promising start to back test the strategy.

Using the above findings, we developed a event driven code, with this parameters

Long criteria –

- Stock drops more than 4%
- The drop must happen 9:30 am to 10:30 am
- We go long when ema (5) crosses sma (10)
- Stop loss – 2% below the lower Bollinger band price level at the time of long.
- If stop loss or profit criteria misses, close the trade at the end of day.

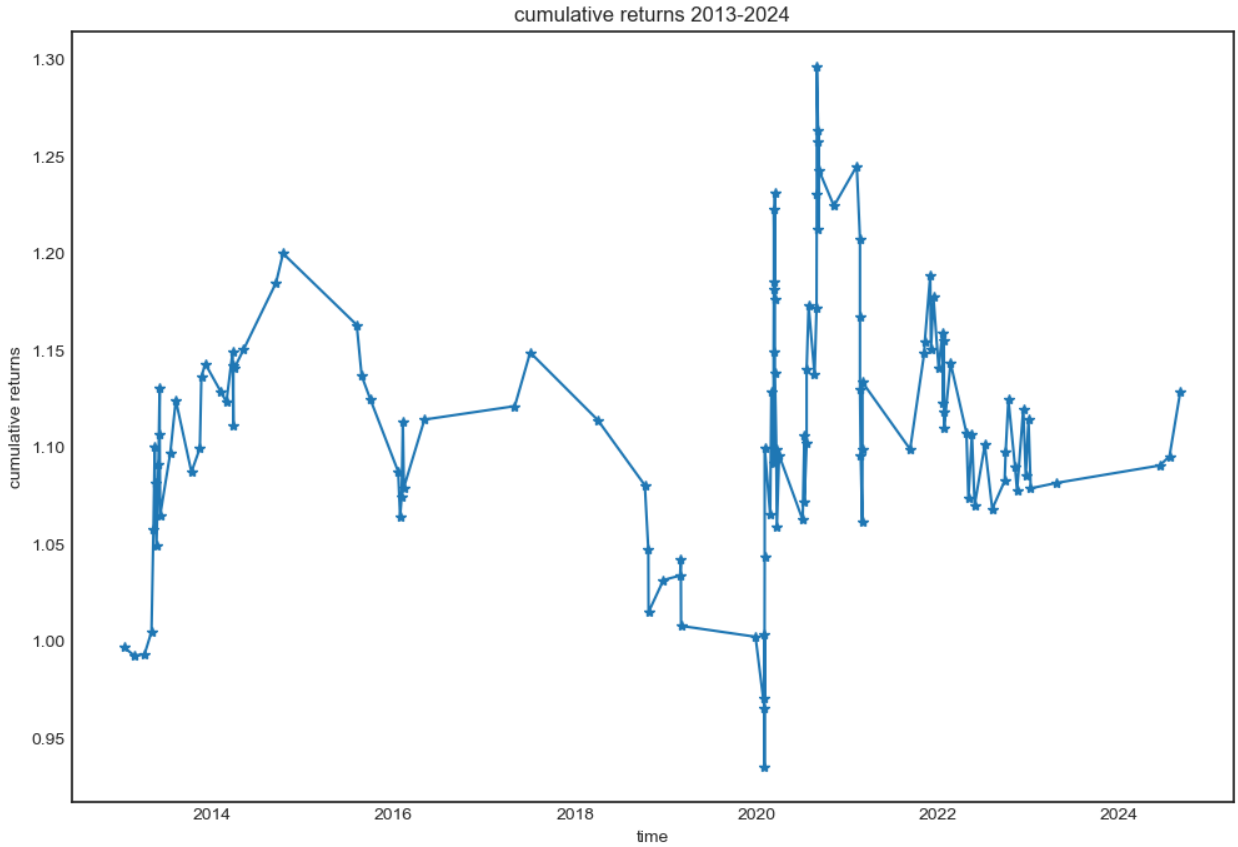
The below shows the output file generated using the event driven code

	entry_price	stop_loss	stop_loss_hit_price	profit_taken_price	signal_time	stop_loss_time	profit_time	close_price	close_time
0	2.1853	2.1061310061440714			2013-01-11 9:55			2.1933	2013-01-11 15:59
1	2.334	2.270321935507273			2013-02-21 9:56			2.3439	2013-02-21 15:59
2	2.742	2.6512902009822112			2013-04-03 10:11			2.74	2013-04-03 15:59
3	3.592	3.4934581303900987			2013-05-01 10:00			3.55	2013-05-01 15:59
4	4.5916	4.351581561	4.3485		2013-05-09 9:40	2013-05-09 10:27			
5	6.146	5.915868655414139	5.902		2013-05-14 9:59	2013-05-14 10:20			
6	6.0467	5.851748015784438			2013-05-16 9:53			6.1467	2013-05-16 15:59
7	5.7625	5.576795881247146		5.9367	2013-05-21 10:19		2013-05-21 14:11		
8	7.059	6.776550410734561	6.7753		2013-05-29 10:13	2013-05-29 10:31			
9	6.1267	5.942427214	5.904		2013-06-03 9:53	2013-06-03 10:04			
10	6.044	5.7593887405090465			2013-06-03 10:09			6.1727	2013-06-03 15:59
11	6.0653	5.885927344016872		6.2953	2013-06-05 10:05		2013-06-05 14:38		
12	8.0789	7.844041131114624	7.8333		2013-07-16 10:05	2013-07-16 10:55			
13	9.1753	8.834114883			2013-08-07 9:45			8.9487	2013-08-07 15:59
14	11.05	10.537830255813322		11.4147	2013-10-09 9:54		2013-10-09 13:24		
15	9.2953	9.069351854			2013-11-12 10:16			9.19	2013-11-12 15:59
16	8.674	8.384229106	8.3827		2013-11-18 10:07	2013-11-18 12:09			
17	9.316	9.034247074			2013-12-04 9:59			9.262	2013-12-04 15:59
18	11.4827	11.162825353471835			2014-02-05 10:29			11.6287	2014-02-05 15:59
19	16.7593	16.24825295624855			2014-02-27 10:21			16.8353	2014-02-27 15:59
20	14.7793	14.443578810472816	14.438		2014-03-24 9:46	2014-03-24 10:35			
21	13.7789	13.291552855524563		14.2339	2014-03-27 9:48		2014-03-27 10:26		
22	14.1347	13.748052720123138	13.7427		2014-03-27 10:27	2014-03-27 11:32			
23	13.8813	13.532674454575503			2014-03-31 10:21			13.898	2014-03-31 15:59
24	13.5326	13.038596346253035			2014-05-07 10:04			13.4187	2014-05-07 15:59
25	17.7073	17.192652090658154	17.1761		2014-09-15 9:57	2014-09-15 11:16			
26	15.1647	14.644692260017326			2014-10-13 9:58			14.9713	2014-10-13 15:59
27	15.962	15.555730740626686		16.4527	2015-08-06 10:04		2015-08-06 10:53		
28	14.6627	14.200186674289963			2015-08-26 10:23			14.994	2015-08-26 15:59
29	16.3847	15.828893962188717			2015-09-30 9:51			16.56	2015-09-30 15:59
30	12.873	12.501998856998805		13.2993	2016-01-20 10:09		2016-01-20 13:51		
31	12.3826	12.08215003330524			2016-01-28 10:14			12.6467	2016-01-28 15:59
32	11.678	11.305079440958725			2016-02-03 9:52			11.5653	2016-02-03 15:59
33	10.1953	9.839952176482008	9.8267		2016-02-08 9:42	2016-02-08 13:12			
34	9.75	9.422754391397907		10.0493	2016-02-12 10:20		2016-02-12 11:26		
35	14.6647	14.217501294163913	14.188		2016-05-05 9:49	2016-05-05 10:37			
36	19.814	19.338549247514713			2017-05-04 10:08			19.6907	2017-05-04 15:59
37	22.3536	21.747290147532702			2017-07-05 10:11			21.8013	2017-07-05 15:59
38	17.386	16.790231090652995		17.913	2018-04-03 9:53		2018-04-03 13:49		
39	16.7189	16.176907012020152		17.225	2018-10-10 10:08		2018-10-10 11:49		
40	19.3267	18.69565728659506		19.918	2018-10-24 9:53		2018-10-24 12:26		
41	20.2727	19.698143041435515		20.9	2018-10-25 10:25		2018-10-25 14:25		
42	20.0193	19.39236730407247			2018-12-24 9:55			19.6927	2018-12-24 13:00

Figure 18 output of the event driven code

The whole code can be accessed using event_driven_tsla.py.

Results of the back test



Our back test results show that overall our strategy has shown 12% return over the entire period of 2013- 2024 .

Total trades done in the entire period: 122

Winning trades: 62

Losing trades: 60

Mean return per trade: 0.0014639930879452984

Max return on a trade: 0.07505350435314327

Max loss on trade: -0.04438196829541493

Overall we see positive results however we see strategy fails to beat the benchmark over the same period.

Challenges / limitations

The strategy assumes that uptrend or downtrend does not affect the intraday movement of the stock. It also disregards the behaviour of tesla in different phases of the market for example the consolidation phase, the bull phase or the black swan events like covid 19. Some of the other assumptions we have seen is the stop loss level we have intentionally kept stop loss level 2* lower Bollinger band level. This stop loss level is entirely based on the judgement level, more could be done to optimise the stop level.

Secondly this strategy does not take into consideration the volatility of the pnl while in the trade this could have great implication on the bottom line if not taken care of.

Lastly this strategy could be improved by seeing the returns of the strategy in the various trend's regimes. Fore example does strategy perform better when it is in uptrend or downtrend.

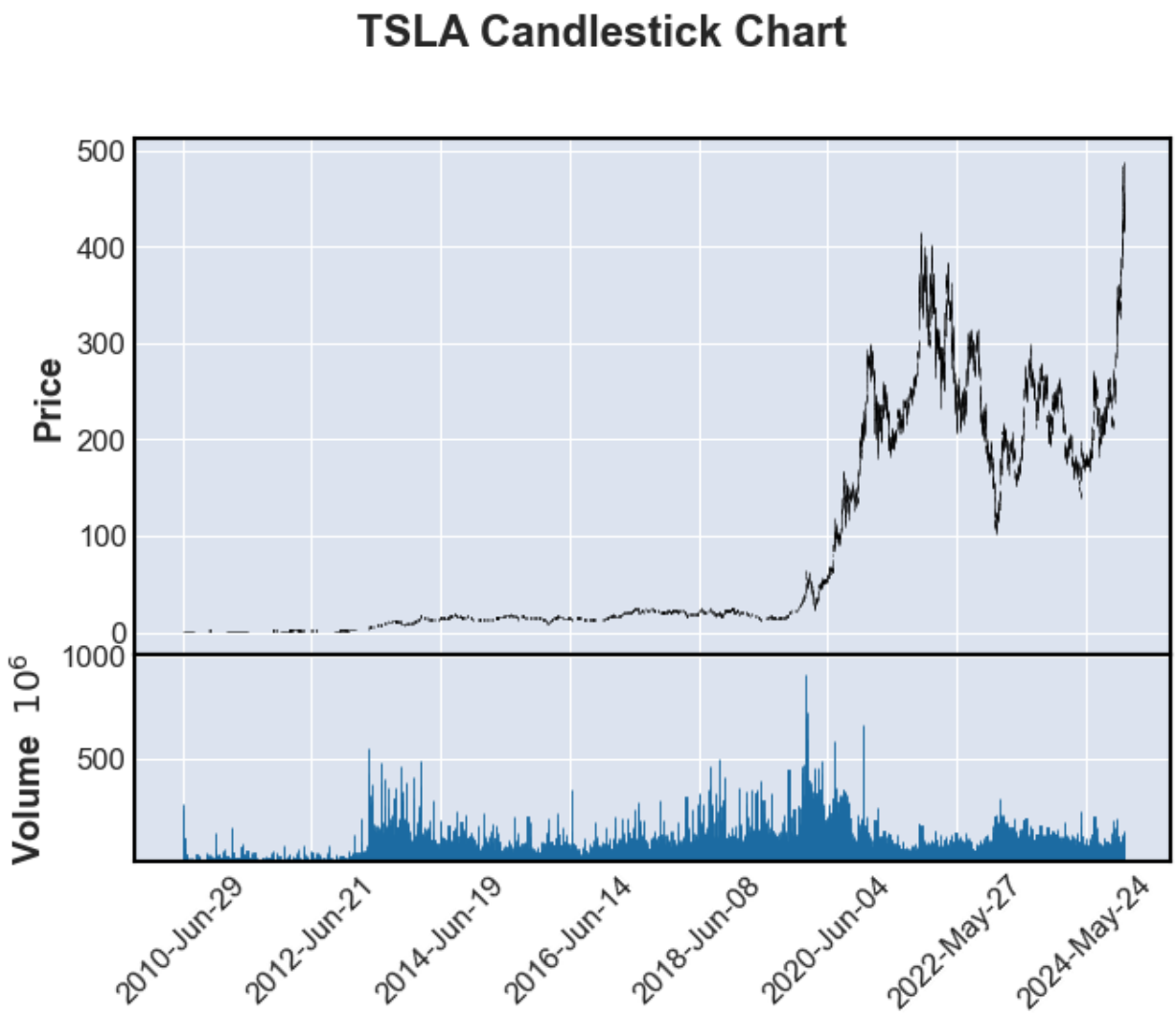


Figure 19 shows the volume profile and price of tsla over the period of time

In the above chart we can see Tesla has had significant volume over the 2020-2024 period. This should also be taken into consideration regarding the performance of the strategy.

Conclusion

Our strategy overall shows positive results but fails to beat benchmark. More refinement and conditions need to be considered to improve the bottom line of the strategy.

Annexure / codes

Tsla_range computation_.py → will give yearly stats as to how many times our condition of 4% drop were met . one can use this file as per the use , we can input combined 1 min data for 2013-2024, or put yearly data in it.

Tsla_utility_functions.py – has multiple functions→ returns distribution() plots the qq plot , jarque bera test.

Plot chart()→function plots interactive candlestick and volume chart for any time period

Calculate momentum()→ computes statistics for uptrend and downtrend candles

Tsla_combine_yearly.py→ will combine all the yearly minute csv data into one.

Event_driven_tsla.py → will compute the strategy as presented in the introduction

Tsla_evaluation_results.py→ computes statistics on the trades data delivered by the event driven code.

Csv files – Tsla_combined _data→ contains 3 csv files namely Trades_tsla which is output of the event driven code.

Tsla_combined_data which is the input to the event driven code

Tsla_stats_data → which is the result of the range computation file.